

CSE 4345: Big Data Analytics

Week 2 Lecture Notes

Challenges of Big Data — Extraction, Integration, and Heterogeneous Information

Department of Computer Science & Engineering | Premier University, Chittagong

CLO: CLO2 — Analyse the existing challenges in big data **PLO:** PLO(b), PLO(c)

After studying these notes you will be able to:

- Explain why the data integration problem exists and describe the three types of schema heterogeneity
- Define schema mapping, data exchange, and entity resolution and explain their role in integration pipelines
- Identify and analyse the five core big data integration challenges: heterogeneity, scale, noise, provenance, and latency
- Define the six dimensions of data quality and describe mitigation strategies for each
- Distinguish between ETL and ELT architectures, explain when each is appropriate, and trace a query through both pipelines
- Compare three data integration architectures: data warehouse, federated query system, and data lake
- Analyse the healthcare interoperability case study (HL7/FHIR) as an instance of all five integration challenges

1. The Data Integration Problem: Why It Exists

Week 1 established that big data is characterised by Volume, Velocity, and Variety. Of these three Vs, **Variety is arguably the hardest engineering challenge** because it is not just a matter of size — it is a matter of meaning.

Organisations rarely collect data from a single clean source. A large hospital collects patient records from its own EHR system, lab results from a separate LIMS, billing data from a third-party system, and radiology images from a PACS. A retailer receives sales from a point-of-sale terminal, web clicks from an analytics platform, inventory levels from an ERP, and returns via a customer-service portal. Each of these systems was built independently, at different times, by different vendors, with different assumptions about what data represents and how it should be structured.

The Data Integration Problem

Given a set of **heterogeneous, distributed data sources**, provide users with a **unified, coherent view** that enables them to query and analyse the combined data as if it came from a single, well-structured system.

(Halevy, Rajaraman & Ordille, 2006 — the canonical framing of the problem)

The difficulty is not just technical. It is semantic: the word “patient” means something slightly different in an orthopaedics system versus a mental-health record system. The field “date of birth” may be stored as DD/MM/YYYY in one system and as a Unix timestamp in another. Integration must resolve these conflicts without losing information or introducing errors.

1.1 Data Silos: The Root Cause

A **data silo** is an isolated repository of data that is accessible only to one team or system and cannot be easily shared with others. Silos arise for organisational reasons (different departments build their own tools), technical reasons (legacy systems with proprietary APIs), and regulatory reasons (patient data must remain within the hospital’s own servers).

Example: Bangladesh Government Data Silos

Bangladesh’s Ministry of Health and Family Welfare (MOHFW) operates several independent databases: the DGHS health facility registry, the DGDA drug licensing database, the BDRIS civil registration system (births and deaths), and the NID database at Bangladesh Election Commission. A public health researcher who wants to correlate NID demographic data with DGHS disease records must navigate four different agencies, four different data formats, and four different legal access regimes. This is the integration problem at a national scale.

2. Three Types of Schema Heterogeneity

When two data sources describe the same real-world entities, their schemas often differ in three distinct ways. Understanding these distinctions is essential because each requires a different technical remedy.

2.1 Structural Heterogeneity

Structural heterogeneity (also called **schematic heterogeneity**) occurs when two sources model the same information using a **different data structure**.

Manifestation	Example	Remedy
Different attribute names for the same concept	Source A: <code>dob</code> ; Source B: <code>birth_date</code>	Schema mapping (rename)
Decomposition vs. composition	Source A: <code>full_name</code> ; Source B: <code>first_name</code> + <code>last_name</code>	Transformation function
Nesting level	Source A: flat CSV row; Source B: nested JSON object	Flattening / structuring
Attribute representation	Source A: boolean <code>is_active</code> ; Source B: string <code>status</code> IN ('active', 'inactive')	Domain mapping
Missing attributes	Source A has <code>phone</code> ; Source B has no <code>phone</code> field	Fill with NULL; document in meta-data

2.2 Semantic Heterogeneity

Semantic heterogeneity is deeper and harder to resolve automatically. It occurs when two sources use **the same term to mean different things** (homonyms) or **different terms to mean the same thing** (synonyms).

- **Synonym problem (different terms, same meaning):** Source A calls a field `patient_id`; Source B calls it `case_number`; Source C calls it `nhsid`. All three refer to a unique identifier for the same person in the healthcare system.
- **Homonym problem (same term, different meaning):** The word **bank** means a financial institution in one database and the bank of a river in another. More subtly, **revenue** may mean gross revenue in one system and net revenue after returns in another — the same column name, different calculation, silent inconsistency.
- **Granularity conflict:** Source A stores temperature in Celsius; Source B in Fahrenheit. Source A reports sales at the transaction level; Source B at the daily aggregate level.
- **Temporal conflict:** Source A records “current address” (may change); Source B stores address as of registration date (historical, immutable).

Common Misconception: Semantic heterogeneity cannot be resolved by software alone. It requires **domain experts** (clinicians, accountants, legal staff) who understand what the fields actually mean. Data engineers who skip this step produce silently wrong integrated outputs — the most dangerous kind of error.

2.3 Syntactic Heterogeneity

Syntactic heterogeneity refers to differences in **physical encoding** and **file format**, even when the structure and semantics are the same.

- **Date formats:** 12/06/2024 (USA: December 6), 06/12/2024 (UK/BD: 6 December), 2024-06-12 (ISO 8601)
- **Character encoding:** UTF-8 vs. ISO-8859-1 vs. Windows-1252 — critical for Bangla text () which requires Unicode
- **Decimal notation:** 1,500.75 (US) vs. 1.500,75 (German/European) vs. 1,500.75 (BDT with comma thousands separator)
- **File format:** CSV, JSON, XML, Parquet, ORC, Avro, fixed-width flat file, EBCDIC (mainframe legacy)
- **Null representation:** empty string, NULL, N/A, -1, 0, or simply absent field in JSON

Key Insight In practice, syntactic heterogeneity is the easiest to fix — parsers, decoders, and format converters handle it mechanically. Structural heterogeneity requires schema mapping rules. Semantic heterogeneity requires human expertise and ontologies. Projects routinely underestimate the time spent on the semantic layer.

3. Seminal Research: Halevy, Rajaraman & Ordille (2006)

The landmark paper “*Data Integration: The Teenage Years*” (PVLDB 2006) reviewed three decades of data integration research and distilled the core technical problems into three distinct sub-problems. Understanding these is essential because every modern integration tool — from Apache Nifi to AWS Glue to dbt — addresses one or more of them.

3.1 Schema Mapping

Schema Mapping

A schema mapping is a formal specification of how the elements of a **source schema** correspond to elements of a **target schema**. It defines the rules by which data is translated from one representation to another.

Schema mapping is expressed as a set of rules of the form: “*For every record in source table S matching condition C , produce a record in target table T with fields computed by expressions $E_1, E_2, \dots$* ”

Two classical paradigms for schema mapping:

1. **Local-as-View (LAV):** Each source is defined as a *view* over a global canonical schema. New sources are easy to add (just write a new view), but answering a query

over the global schema requires unfolding the views — computationally expensive. Used by: federated query systems (Apache Drill, Presto with multiple connectors).

2. **Global-as-View (GAV):** The global schema is defined as a *view* over the sources. Queries over the global schema are easy to answer, but adding a new source requires rewriting the global view. Used by: traditional data warehousing (the canonical star-schema ETL model).

Example: Schema Mapping in a Hospital Integration

A national health registry defines a canonical `Patient` schema with fields: (`national_id`, `full_name`, `dob`, `gender`, `blood_type`, `primary_diagnosis_icd10`). Hospital A stores patients as (`patient_code`, `name_first`, `name_last`, `date_birth`, `sex`, `icd10_code`). The schema mapping rule is:

```
national_id ← patient_code
full_name ← CONCAT(name_first, ' ', name_last)
dob ← TO_DATE(date_birth, 'DD/MM/YYYY')
gender ← CASE WHEN sex='M' THEN 'Male' ELSE 'Female' END
```

3.2 Data Exchange

Data Exchange

Data exchange is the problem of **materialising** (physically computing and storing) a target database that satisfies a given schema mapping from a source database. Unlike schema mapping (which just specifies correspondence), data exchange actually transforms and moves the data.

Data exchange is the core operation in ETL pipelines. The source is read, the mapping rules are applied row by row or in bulk, and the results are written into the target schema. Key challenges:

- **Completeness:** What to do when a source record has no corresponding target record (orphaned foreign keys, missing lookups)?
- **Null propagation:** When a source field has no target equivalent, how is that loss documented?
- **Consistency:** If the same logical entity appears in two sources and the values conflict, which source “wins”?
- **Incrementality:** At petabyte scale, re-exchanging the entire dataset on every pipeline run is prohibitive. Incremental (CDC-based) exchange is required.

3.3 Entity Resolution

Entity Resolution (ER)

Entity resolution (also called **record linkage**, **deduplication**, or **data matching**) is the process of determining whether two records from the same or different datasets refer to the **same real-world entity**, even though they may not share a common identifier.

This is the most computationally challenging of the three sub-problems. The naive approach — compare every record in Source A with every record in Source B — has complexity $O(|A| \times |B|)$. For two datasets of 10 million records each, this is 10^{14} comparisons: infeasible.

Three-stage entity resolution pipeline:

1. **Blocking (Candidate Generation):** Reduce the comparison space by grouping records that *could* match. Only records in the same block are compared. Common blocking keys: first three letters of surname + birth year, postcode + first letter of name, phonetic code (Soundex, Metaphone) of name.
2. **Similarity Computation:** For each candidate pair within a block, compute a similarity score using one or more metrics:
 - **Edit distance (Levenshtein):** Number of character insertions, deletions, or substitutions needed to transform one string into another. “Mohammed” vs. “Mohammad” = edit distance 2.
 - **Jaccard similarity:** Overlap of token sets. $J(A, B) = |A \cap B| / |A \cup B|$. Good for name variants with different word order.
 - **Cosine similarity:** Used for longer text fields (addresses, notes).
 - **Date proximity:** If DOB differs by ≤ 365 days, give partial credit (accounting for data entry errors).
3. **Classification (Match / Non-match / Possible):** Based on the similarity scores, classify each candidate pair as a definite match, definite non-match, or uncertain case for manual review. Threshold-based rules (“if *Levenshtein* < 3 AND same DOB $\rightarrow$ MATCH”) or supervised ML classifiers (logistic regression, random forests) trained on labelled pairs.

Deep Dive: Why Entity Resolution Is Hard in Practice

Consider integrating bKash mobile wallet records with Bangladesh Bank transaction logs. bKash stores users as (`phone_number`, `name`). Bangladesh Bank stores entities as (`account_number`, `NID_number`, `name`). There is no shared key. Furthermore:

- Names in Bangladesh are commonly misspelled in different systems: “Mohammad Rafiqul Islam” in one system, “Md. Rafiqul Islam” in another, “Rafiqul Islam” in a third
- One person may have multiple phone numbers (work, personal, family member’s number used for bKash)
- NID numbers may contain OCR errors from manual data entry at union offices

A production ER system for this scenario would use blocking on phone-number prefix + first two characters of name, Levenshtein distance on name, and a logistic regression classifier trained on 10,000 manually labelled pairs. Even so, a 95% precision, 90% recall outcome would still incorrectly link tens of thousands of records at national scale.

4. The Five Big Data Integration Challenges

Beyond the three classical sub-problems of integration research, big data introduces five additional engineering challenges that did not exist (or existed in milder form) in traditional data warehousing.

4.1 Challenge 1: Heterogeneity

Heterogeneity is the multiplicity of formats, schemas, and semantics described in Section 2. At big data scale, it is amplified:

- A single organisation may have **hundreds of source systems**: legacy COBOL files, modern REST APIs, Kafka streams, flat files on FTP servers, real-time IoT sensors, and PDF documents scanned by OCR
- Data lakes ingest **without transformation** at write time (schema-on-read), meaning all the heterogeneity is preserved and must be resolved at query time
- Machine-generated data (IoT sensors, server logs) has implicit schemas that must be inferred or documented separately

4.2 Challenge 2: Scale

Scale creates problems not just of storage but of **pipeline throughput and latency**:

- Traditional ETL tools (Informatica, SSIS) were designed for terabyte-scale warehouses running on single powerful servers. At petabyte scale they become bottlenecks.
- A single JOIN between two 10 TB tables requires moving data across the network or using a distributed join algorithm. Neither is trivial.
- Index structures that work well at small scale (B-trees) become impractical at petabyte scale — full-table scans with predicate pushdown (Parquet/ORC) replace them.

- **Backfill problem:** When a new data requirement is added (e.g., a new column must be computed for all historical records), reprocessing years of data at petabyte scale takes days or weeks.

Operation	Traditional (single-server)	Big Data approach
Load 1 TB of CSV	Hours, limited by server disk I/O	Parallel ingest across 100 nodes (Sqoop, Flume)
Join two 5 TB tables	Impossible in memory; temp-file based	Distributed sort-merge join in Spark/Hive
Recompute one column for 10 TB history	Days on a single machine	Hours on a Spark cluster with partition pruning
Run 100 concurrent queries	Query queue degrades; resource starvation	YARN/K8s with capacity scheduler

4.3 Challenge 3: Noise and Missing Values

Real-world data is never perfectly clean. In big data pipelines, data quality problems are **amplified by volume**: even a 0.1% error rate across 1 billion records leaves 1 million corrupted records in the integrated dataset.

Common sources of noise and missing values:

- **Sensor dropout:** IoT sensors miss readings due to network outages, battery failure, or hardware failure. A temperature sensor that sends one reading per minute may produce time-series data with irregular gaps.
- **OCR errors:** Physical documents scanned with optical character recognition introduce systematic errors: 0 ↔ O, I ↔ 1, S ↔ 5, especially in low-quality scans from government offices with ageing infrastructure.
- **Voluntary non-response:** Survey respondents skip sensitive questions (income, health conditions), producing systematic rather than random missing data — biasing downstream analytics.
- **System-generated defaults:** Developers often insert NULL, 0, or 9999-12-31 as placeholders for missing values. If not documented, these pollute aggregate calculations (average age of 9999-year-olds is not useful).
- **Encoding cascades:** A file transmitted from a UTF-8 system and received by a Windows-1252 system corrupts Bangla characters into garbled byte sequences (â€™ instead of an apostrophe is a classic encoding cascade artefact).

4.4 Challenge 4: Provenance

Data Provenance

Data provenance (also called **data lineage**) is the **documented history** of where a piece of data came from, how it was transformed, and who accessed it. It answers the questions: *Where did this value originate? What processing steps produced the current form? Who approved its use?*

Provenance is important for four distinct reasons:

1. **Regulatory compliance:** The EU General Data Protection Regulation (GDPR, 2018) requires organisations to document what personal data they hold, where it came from, and what they do with it. Bangladesh’s Personal Data Protection Act (under development) will impose similar requirements. Failure to demonstrate provenance can result in substantial fines.
2. **Debugging:** When a derived metric (e.g., monthly active users) suddenly changes by 30%, provenance lets engineers trace the value back through the pipeline to find which upstream source, transformation step, or schema change caused the anomaly.
3. **Auditability:** In financial systems, every record in a general ledger must be traceable to a source transaction. A bank auditor must be able to follow a value from the consolidated balance sheet back to the originating branch transaction without ambiguity.
4. **Reproducibility:** In scientific analytics (genomics, clinical trials), a finding is only valid if the exact same analysis run on the same data produces the same result. Provenance records the data version, pipeline version, and parameters used.

Example: Provenance Failure at a Bangladesh Utility Company

A DESCO (electricity utility) billing system draws consumption data from smart meters, applies tariff rules, and produces monthly bills. If a customer disputes a bill, the customer service team must trace: (1) which meter reading was used, (2) when it was collected, (3) which tariff table version was active at billing time, (4) whether any manual adjustments were applied, and by whom. Without documented provenance, this trace is impossible — leading to arbitrary resolutions, customer complaints, and regulatory exposure.

4.5 Challenge 5: Latency

Latency in data integration refers to the **delay between when data is generated and when it is available for analysis** in the integrated system.

- **Batch latency:** A nightly ETL job that runs at 02:00 means integrated data is always at least 14–26 hours old. A decision-maker querying the dashboard at 15:00 is looking at yesterday’s reality.
- **Pipeline lag:** Even in “near real-time” streaming pipelines, each stage adds latency: Kafka topic lag, transformation processing time, write commit time, and index update

time. Total end-to-end latency for a production streaming pipeline is typically 1–30 seconds.

- **Late-arriving data:** Events generated in one time zone, on mobile devices with intermittent connectivity, or by offline sensors may arrive hours or days after the fact. A fraud detection system must decide whether to reprocess historical windows when late events arrive, or simply discard them.
- **Consistency lag:** When data is replicated across geographically distributed data centres (Dhaka, London, Singapore), changes made in one data centre take time to propagate to others. During this propagation window, different users may see different versions of the same data — the core trade-off described by the CAP theorem from Week 1.

5. Data Quality: Dimensions and Mitigation Strategies

Data quality is not a binary property (“clean” vs. “dirty”). It is measured across **six dimensions**, each of which can be assessed and improved independently.

Dimension	Definition	Example Violation	Mitigation
Completeness	All required fields are present and populated	Missing <code>blood_type</code> for 40% of patients	Imputation; mark as unknown; reject at ingest
Accuracy	Values correctly reflect the real-world state	Age of 150 due to wrong birth year entered	Range checks; cross-validation with NID
Consistency	No contradictions within or across datasets	Discharge date before admission date	Constraint checks; referential integrity rules
Timeliness	Data is available when needed and is current	Exchange rates updated only weekly in a real-time trading system	Reduce pipeline latency; SLA monitoring
Uniqueness	Each entity appears exactly once	A customer registered twice with two email addresses	Entity resolution (deduplication)
Validity	Values conform to defined business rules and formats	Phone number “abc123” in an integer field	Schema validation; regex checks at ingest

Deep Dive: The Data Quality Iceberg

Most teams discover data quality problems *after* a business decision has already been made on bad data. The classic warning sign is an analyst who says “the numbers don’t add up” — meaning two reports derived from the same data source disagree. The root cause is almost always one of the six dimensions above. Best practice is to build **data quality checks as automated pipeline steps**, not as after-the-fact audits. Tools like **Apache Griffin**, **Great Expectations**, and **dbt tests** allow you to define quality rules (e.g., “*no more than 2% nulls in the age column*”) and fail the pipeline if they are violated — catching problems before they reach analysts.

5.1 Data Quality in Bangladesh: Practical Context

Bangladesh’s government datasets, while increasingly digitised, face persistent data quality challenges rooted in the transition from paper to digital record-keeping:

- **Name variability:** Bangladeshi names are commonly recorded in multiple translit-

erations of Bengali script into Roman script (“Rahim”, “Raheem”, “Rahimuddin”). There is no standardised official romanisation.

- **NID errors:** Manual entry at union parishad level introduced systematic OCR and transcription errors in early NID batches (pre-2016 smart cards). Date of birth discrepancies between NID and birth certificate are common.
- **Address inconsistency:** Address fields are unstructured free text across most legacy government databases — no postcode standard, no geocoding. The same building may appear as “House 12, Road 4, Dhanmondi” in one system and “12/4 Dhanmondi” in another.

6. ETL vs. ELT: Two Paradigms for Moving Data

Once data sources are understood and quality issues are identified, the data must be physically moved into an integrated store for analysis. There are two dominant architectural patterns for this.

6.1 Traditional ETL (Extract — Transform — Load)

ETL (Extract-Transform-Load)

In ETL, data is **extracted** from source systems, **transformed** (cleaned, mapped, enriched, aggregated) in a staging area or dedicated ETL server *before* it reaches the target system, and then **loaded** into the target data warehouse in its final, query-ready form.

Step-by-step trace of a classic ETL pipeline:

1. **Extract:** A scheduled job connects to source systems (hospital RDBMS, pharmacy CSV exports, lab LIMS via REST API) and pulls changed records since the last run. Extraction uses bulk reads, database CDC (Change Data Capture) logs, or direct SQL queries. The raw data is written to a **staging area** — a temporary store that is not yet the warehouse.
2. **Transform:** The staging data is processed by a series of transformation steps:
 - *Cleanse:* remove duplicates, fix encoding issues, impute or flag missing values
 - *Standardise:* convert dates to ISO 8601, normalise phone numbers to +880-1x-xxxx-xxxx format
 - *Map:* apply schema mapping rules (rename columns, split/merge fields, join with lookup tables)
 - *Enrich:* add derived columns (age computed from DOB, district name joined from postcode lookup)
 - *Aggregate:* pre-compute summaries if the warehouse uses a star schema
3. **Load:** The fully transformed, validated records are loaded into the target data warehouse (e.g., an Oracle DW, Amazon Redshift, or a Hive ORC table). The load may be a full refresh (truncate-and-reload) or an incremental merge (UPSERT).

Advantages of ETL:

- Data in the warehouse is always in a known, clean, query-ready state
- Analysts do not need to understand raw source schemas
- Performance-optimised (pre-aggregated, indexed, partitioned)

Disadvantages of ETL:

- Transformation logic is a bottleneck: if requirements change, re-running the full ETL is expensive
- Raw data is discarded after transformation — if a new business question arises that needs the original raw format, it is gone
- Does not scale horizontally: ETL servers have a memory limit; processing 100 TB in one transformation step is impractical
- Long development cycle: data engineers must define the transformation schema *before* data scientists explore the data

6.2 Modern ELT (Extract — Load — Transform)

ELT (Extract-Load-Transform)

In ELT, raw data is **extracted** from sources and **loaded immediately** into a scalable distributed store (data lake on HDFS or S3) in its original format. **Transformation** happens *on demand* at query time, or as a post-load computation using distributed engines like Spark or Hive.

Why ELT became possible — and preferred — with big data platforms:

- **Storage is cheap:** HDFS or S3 object storage costs pennies per GB — there is no reason to discard raw data. Raw files are retained indefinitely.
- **Compute is elastic:** A Spark cluster can be provisioned on demand to run transformation jobs. You only pay for compute time, not idle hardware.
- **Schema-on-read:** Hive, Spark SQL, and Presto can impose any schema on a raw file at query time. The same raw web log can simultaneously be queried as a session analytics table and as a security audit table with different schemas.
- **Exploration before commitment:** Data scientists can explore raw data with Spark notebooks *before* any schema is finalised. Once they know what transformations are needed, those are codified as a scheduled dbt or Spark job — not designed upfront by a data engineer.

Property	ETL	ELT
Transform timing	Before loading (in staging)	After loading (on demand)
Raw data retained?	No (transformed state only)	Yes (raw data preserved forever)
Schema defined when?	Before data is loaded	At query time (or as a post-load step)
Suitable data store	Relational data warehouse (Redshift, Oracle DW)	Data lake (HDFS, S3, Azure ADLS)
Processing engine	ETL tool (Informatica, SSIS, Talend)	Spark, Hive, dbt, Presto
Scalability	Limited (single ETL server)	Horizontal (Spark cluster)
Exploration flexibility	Low (schema fixed upfront)	High (explore raw data first)
Compliance/auditability	Harder (raw data gone)	Easier (full history preserved)
Best for	OLTP-adjacent warehouses, strict SLAs	Data lakes, ML pipelines, exploratory analytics

Common Misconception: ELT does not eliminate transformation — it defers it. The risk is that transformation logic becomes scattered across dozens of ad-hoc Spark notebooks with no governance, producing inconsistent results for the same business question across different teams. Tools like **dbt (data build tool)** restore discipline by defining transformations as versioned, tested SQL models on top of a data lake, combining ELT's flexibility with ETL's governance.

7. Data Integration Architectures

At an architectural level, there are three fundamental approaches to integrating heterogeneous data sources. Each makes a different trade-off between query performance, data freshness, transformation flexibility, and operational complexity.

7.1 Architecture 1: Data Warehouse (Centralised, Pre-Integrated)

In a data warehouse architecture, all source data is **periodically extracted, transformed, and loaded** into a single, centralised, schema-optimised store. Analysts query the warehouse, not the sources.

Canonical structure: the Star Schema

- **Fact table:** contains measurable, quantitative events (sales transactions, hospital admissions) with foreign keys to dimension tables
- **Dimension tables:** contain descriptive attributes (patient demographics, product catalogue, date hierarchy, location hierarchy)
- **Why “star”?:** The fact table in the centre is connected to dimension tables at the points — the ER diagram looks like a star

Strengths: Fast, predictable query performance; clean, validated data; familiar SQL interface for analysts.

Weaknesses: Rigid schema makes it hard to add new data types; raw data is discarded; nightly ETL means data is always stale; unstructured data (images, free text) cannot be stored.

7.2 Architecture 2: Federated Query System

In a federated architecture, data **remains in its original source systems** and is never centralised. A **middleware query layer** accepts SQL queries, decomposes them into sub-queries for each relevant source, executes the sub-queries in parallel, and assembles the results.

Examples: Apache Drill, Presto with multiple connectors (can query HDFS + MySQL + Cassandra in a single JOIN), Google BigQuery Omni.

Strengths: Data always current (no ETL lag); no duplication; sources retain autonomy.

Weaknesses: Performance is bounded by the slowest source; complex optimisation required; sources must be available at query time; cannot aggregate across sources that do not expose a query interface.

7.3 Architecture 3: Data Lake (Modern ELT)

A data lake stores raw data of all types (structured, semi-structured, unstructured) in a distributed object store (HDFS, Amazon S3, Azure ADLS), at any scale, without requiring upfront schema definition. Downstream consumers apply their own schemas at query time.

Strengths: Petabyte-scale; preserves raw data indefinitely; supports all data types; enables exploratory analytics and ML.

Weaknesses: Without governance, becomes a “**data swamp**” (data lands but no one can find or trust it); no inherent data quality enforcement; query performance is lower than a pre-aggregated warehouse.

Key Insight Modern production architectures often use **all three** in layers: a data lake (S3/HDFS) for raw ingestion and archival; a curated warehouse layer (Hive ORC tables or Redshift) for governed, validated datasets; and Presto/Spark SQL as the federated query engine on top. This is the “data lakehouse” architecture — covered in Week 11.

8. Case Study: Healthcare Interoperability and HL7 FHIR

The healthcare sector is arguably the most challenging domain for data integration. It exhibits all five integration challenges simultaneously, at scale, with life-critical consequences for failure.

8.1 The Problem: Incompatible EHR Systems

In most countries, hospitals use **Electronic Health Record (EHR) systems** from different vendors: Epic, Cerner, Meditech, Allscripts. Each system stores patient data in a proprietary format. When a patient is transferred from one hospital to another, their medical history *cannot be automatically transferred* — a clinician must request paper records, wait days, and manually re-enter data.

A 2019 Johns Hopkins study found that **integration failures cost US hospitals \$8.3 billion annually** — not primarily in software costs, but in wasted clinician time, duplicate tests, medication errors, and adverse events caused by missing allergy or drug information.

8.2 The Solution: HL7 FHIR

HL7 FHIR (Fast Healthcare Interoperability Resources)

FHIR (pronounced “fire”) is an international standard developed by **Health Level 7 (HL7)** that defines:

- A set of **Resources** (standardised data objects): Patient, Observation, Medication, Condition, Encounter, DiagnosticReport
- A **RESTful API specification**: GET /Patient/{id}, POST /Observation, PUT /Condition/{id}
- **Serialisation formats**: JSON (primary), XML (legacy), Turtle (RDF)
- A **terminology binding** framework linking local codes (ICD-10, SNOMED-CT, LOINC) to canonical international vocabularies

How FHIR addresses the integration challenges:

Challenge	Manifestation in Healthcare	FHIR Remedy
Heterogeneity	Epic uses its own schema; Cerner uses another; lab uses HL7 v2 messages	FHIR Resource schemas are the canonical target; all EHRs publish FHIR APIs
Scale	A national health registry covers millions of patients across thousands of facilities	FHIR servers backed by Elasticsearch or HAPI FHIR (PostgreSQL-based); federated query
Noise / Missing	Allergy field absent in 30% of legacy records	FHIR “must-support” cardinality rules; data quality dashboard per facility
Provenance	Who entered this diagnosis? When? Which version of ICD-10?	FHIR Provenance resource records actor, timestamp, reason, target
Latency	Referral data arrives at receiving hospital after the patient	FHIR subscriptions (push notifications on resource change); near real-time transfer

8.3 FHIR in Bangladesh

Bangladesh’s MOHFW is developing the **DGHS Health Information Management System (HIMS)** with FHIR as its interoperability standard. Key milestones:

- The **BanglaHIE** (Bangladesh Health Information Exchange) initiative, supported by WHO and USAID, is standardising patient identity using NID numbers as the master patient identifier linked to FHIR Patient resources
- District hospitals are being piloted with HAPI FHIR servers that aggregate data from **community health workers** (via mobile app), upazila health complexes, and district hospitals into a single regional FHIR repository
- The primary challenge is **entity resolution**: linking the same patient across community health worker records (keyed on phone number), upazila clinic records (keyed on a local patient ID), and NID (keyed on NID number) — a three-way ER problem across 170 million people

9. Summary and Key Takeaways

Concept	What to Remember
Data integration problem	Providing unified query access to heterogeneous, distributed sources (Halevy et al., 2006)
Three heterogeneity types	Structural (different schemas), Semantic (different meanings), Syntactic (different encodings)
Schema mapping	Formal rules translating source schema to target schema; LAV (federated) vs. GAV (warehouse)
Data exchange	Physical materialisation of data under a mapping; the core operation of ETL
Entity resolution	Determining whether two records refer to the same real-world entity; blocking + similarity + classification
Five challenges	Heterogeneity, Scale, Noise/Missing Values, Provenance, Latency
Six quality dimensions	Completeness, Accuracy, Consistency, Timeliness, Uniqueness, Validity
ETL	Transform <i>before</i> loading; clean data in warehouse; raw data lost; schema rigid
ELT	Load raw first; transform on demand with Spark/Hive; schema flexible; requires governance (dbt)
Data warehouse	Pre-integrated, star schema, fast queries, stale data, no unstructured support
Federated query	Data stays in place; always current; slow for complex joins; sources must be up
Data lake	Raw data, all types, petabyte scale; flexible; risk of becoming a data swamp without governance
HL7 FHIR	RESTful standard for healthcare data exchange; resolves all five integration challenges in healthcare

Key Insight The central lesson of Week 2 is that **getting data is the easy part; making it consistent, trusted, and queryable is hard**. Companies that invest in data integration infrastructure (schema registries, data catalogues, quality pipelines, lineage tracking) build a durable data asset. Companies that skip this step accumulate *technical debt that manifests as wrong business decisions* made from bad data.

10. Self-Assessment

Multiple Choice Questions

Instructions: Select the single best answer.

- Q1.** Which technique determines whether two records from different datasets refer to the **same real-world entity**?
- (a) Schema mapping
 - (b) Data exchange
 - (c) **Entity resolution** ← correct
 - (d) Data partitioning
- Q2.** In the **ELT** paradigm, when does data transformation occur?
- (a) Before extraction from the source
 - (b) During extraction from the source
 - (c) Before loading into the target store
 - (d) **After loading into the target store** ← correct
- Q3.** Which of the following is **NOT** a recognised challenge in big data integration?
- (a) Heterogeneity of source schemas
 - (b) Provenance tracking for compliance
 - (c) **Low network bandwidth at a single computation node** ← correct
 - (d) Data quality noise and missing values

True / False

Instructions: State True or False and write a one-sentence justification for each.

Statement	Answer
1. ETL is preferred over ELT in modern data lake architectures.	False
2. Entity resolution is needed when integrating data from multiple sources about the same real-world object.	True
3. GDPR requires organisations to track the provenance of personal data they process.	True
4. Semantic heterogeneity can be resolved automatically by a schema mapping tool without human domain expertise.	False

Justifications:

1. *False* — Modern data lakes use ELT: raw data is loaded first (cheaply, at high speed), and transformation is deferred until query time using tools like Spark, Hive, or dbt. ETL is the pattern for traditional data warehouses.
2. *True* — When two sources independently store records about the same person, product, or event but use different local identifiers, entity resolution is needed to determine which records refer to the same entity and merge them correctly.
3. *True* — GDPR Articles 13–14 require organisations to inform data subjects about the source of their data; Article 30 requires maintaining a record of processing activities, including data origin and transformation history.
4. *False* — Semantic heterogeneity involves differences in the *meaning* of terms, which requires domain experts (clinicians, accountants, lawyers) who understand the real-world referents. Software tools can automate structural and syntactic mappings but cannot resolve semantic ambiguity without human guidance.

Descriptive Questions

Instructions: Answer each question in 100–150 words.

- D1.** Define the **ETL process** and explain each of the three steps with a concrete big data example drawn from Bangladesh’s financial sector (e.g., bKash, banking, MFS).
- D2.** What is **schema heterogeneity**? Explain the difference between structural heterogeneity and semantic heterogeneity, and give one example of each from a healthcare or government database context.
- D3.** Describe any **three data quality dimensions** from the six covered in these notes. For each dimension, give one specific violation example and one mitigation strategy that could be implemented in an automated data pipeline.

Analytical Questions

Instructions: Answer in 200–300 words with step-by-step reasoning.

A1. Government Healthcare Integration Challenge:

A Bangladesh government agency wants to build a unified national healthcare analytics platform by integrating data from 50 public district hospitals, each using a different EHR system. The data includes structured patient demographics, semi-structured HL7 lab results in XML, and unstructured radiology reports in plain text.

- Identify at least **four** specific integration challenges this project will face (use the five-challenge framework from Section 4)
- For each challenge, propose a concrete technical strategy to address it
- Recommend whether the project should use ETL or ELT, and justify your recommendation

- Identify which of the three integration architectures (warehouse, federated, or data lake) best fits this scenario

A2. ETL vs. ELT Architecture Trade-off:

A Bangladeshi commercial bank runs two systems that both need integrated data from the same transaction sources:

- System A: a nightly regulatory compliance report submitted to Bangladesh Bank by 06:00 each morning, covering all transactions from the previous day in a fixed XML schema
- System B: a real-time fraud detection engine that must decide whether to block a transaction within 200 milliseconds of it being initiated

Analyse how the data integration needs of System A and System B differ across all five challenge dimensions (heterogeneity, scale, noise, provenance, latency). For each system, specify whether you would use ETL or ELT, what execution engine you would use (Spark, Hive, Kafka Streams, etc.), and what the end-to-end data freshness SLA would be.

Textbook References for Week 2

- **[SA]** Seema Acharya & Subhashini Chellappan. *Big Data and Analytics*. Wiley, 2015. **Chapter 2**
Chapter 2 covers sources of big data and the key challenges of collection, storage, and analysis. Includes discussion of data heterogeneity and ETL concepts.
- **[TE]** Thomas Erl, Wajid Khattak & Paul Buhler. *Big Data Fundamentals*. Prentice Hall, 2016. **Chapter 3**
Chapter 3 addresses big data adoption and planning considerations, including data governance, quality frameworks, and integration architectures at the enterprise level.
- **[LRU]** Leskovec, Rajaraman & Ullman. *Mining of Massive Datasets*, 3rd ed. Cambridge, 2020. **Chapter 1.3**
Section 1.3 introduces distributed file systems as the prerequisite for integrating large-scale data, connecting Week 2 to the HDFS architecture in Week 3.
- **Seminal Paper:** Halevy, A., Rajaraman, A., & Ordille, J. (2006). “Data Integration: The Teenage Years.” Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB 2006).
The canonical survey of data integration research. Defines the schema mapping, data exchange, and entity resolution sub-problems that structure Section 3 of these notes. Freely available online.
- **Graduate Text:** Doan, A., Halevy, A., & Ives, Z. (2012). *Principles of Data Integration*. Morgan Kaufmann.
The standard graduate-level textbook on data integration at MIT and Stanford. Chapter 1 (overview) and Chapter 3 (schema mapping) are most relevant to this week’s material.

- **Standard:** HL7 International. *FHIR Release 4*. hl7.org/fhir/R4 (2019).
The normative FHIR specification. The Patient, Observation, and Provenance resource pages are directly relevant to the case study in Section 8.

CSE 4345 — Big Data Analytics | Week 2 Lecture Notes | Department of CSE, Premier University
| Instructor: rufaruqui@cu.ac.bd